Regression & Its Evaluation | Assignment Answers

Question 1: What is Simple Linear Regression?

Simple Linear Regression is a statistical method used to model the relationship between one independent variable (X) and one dependent variable (Y). It predicts the value of Y based on X using a straight-line equation:

$Y = a + bX$

where:

- a = intercept
- b = slope of the regression line

The main goal is to find the best-fit line that minimizes the error between actual and predicted values.

Question 2: What are the key assumptions of Simple Linear Regression?

The key assumptions of Simple Linear Regression are:

1. Linearity – There should be a linear relationship between X and Y.
2. Independence – Observations should be independent of each other.
3. Homoscedasticity – The variance of residuals should remain constant.
4. Normality – Residuals should be normally distributed.
5. No multicollinearity – Independent variables should not be highly correlated.
6. No significant outliers – Extreme values should not heavily influence the model.

Question 3: What is heteroscedasticity, and why is it important to address in regression models?

Heteroscedasticity occurs when the variance of residuals is not constant across all levels of the independent variable. In such cases, prediction errors vary for different values of X.

It is important to address heteroscedasticity because:

- It reduces the reliability of regression coefficients.
- Standard errors become biased.
- Hypothesis testing becomes inaccurate.
- Model predictions may become less reliable.

Common solutions include transforming variables, removing outliers, or using weighted regression.

Question 4: What is Multiple Linear Regression?

Multiple Linear Regression is an extension of Simple Linear Regression that uses two or more independent variables to predict a dependent variable.

The equation is:

$Y = a + b1X1 + b2X2 + ... + bnXn$

It helps analyze how multiple factors influence the output variable simultaneously.

Question 5: What is polynomial regression, and how does it differ from linear regression?

Polynomial Regression is a type of regression where the relationship between variables is modeled using polynomial terms.

Example:
$Y = a + b1X + b2X^2 + b3X^3$

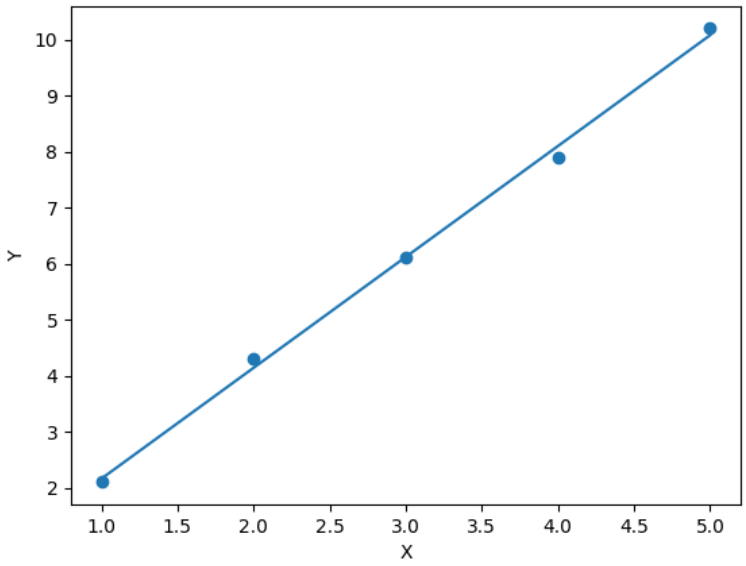
Difference from Linear Regression:
- Linear Regression fits a straight line.
- Polynomial Regression fits a curved line.
- Polynomial Regression is useful when data shows a nonlinear relationship.

Question 6: Implement a Python program to fit a Simple Linear Regression model to the following sample data:

X = [1, 2, 3, 4, 5]
Y = [2.1, 4.3, 6.1, 7.9, 10.2]

Python Code:

```
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2.1,4.3,6.1,7.9,10.2])

model = LinearRegression()
model.fit(X,Y)

pred = model.predict(X)

plt.scatter(X,Y)
plt.plot(X,pred)
plt.show()
```



Question 7: Fit a Multiple Linear Regression model on this sample data:

Area = [1200, 1500, 1800, 2000]
Rooms = [2, 3, 3, 4]
Price = [250000, 300000, 320000, 370000]

Python Code:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from statsmodels.stats.outliers_influence import variance_inflation_factor

data = pd.DataFrame({
   'Area':[1200,1500,1800,2000],
   'Rooms':[2,3,3,4],
   'Price':[250000,300000,320000,370000]
})

X = data[['Area','Rooms']]
y = data['Price']

model = LinearRegression()
model.fit(X,y)

vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
```

```
print(vif_data)
```

VIF Results:

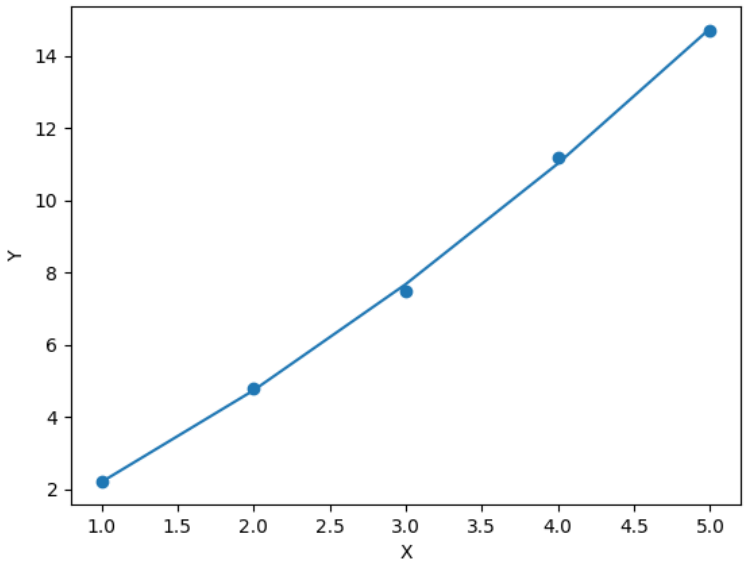
```
Feature       VIF
  Area 127.796923
 Rooms 127.796923
```

Question 8: Implement polynomial regression on the following data:

X = [1, 2, 3, 4, 5]
Y = [2.2, 4.8, 7.5, 11.2, 14.7]

Python Code:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2.2,4.8,7.5,11.2,14.7])

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly,Y)

pred = model.predict(X_poly)

plt.scatter(X,Y)
plt.plot(X,pred)
plt.show()
```

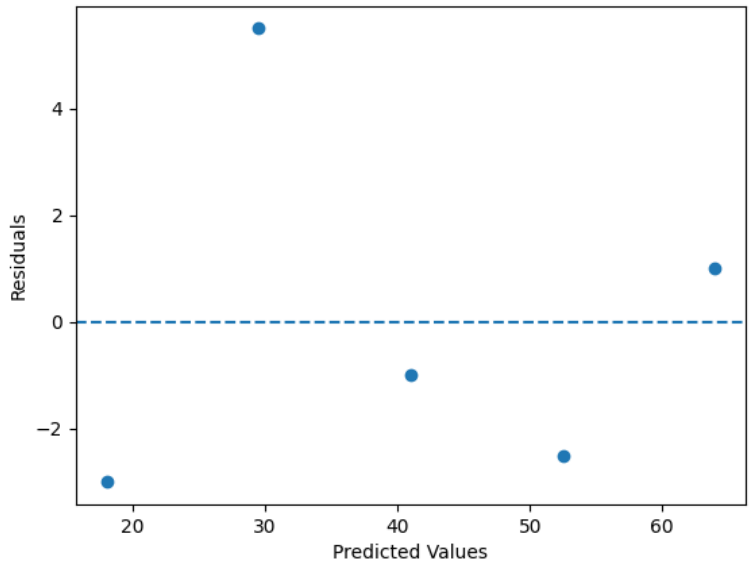


Question 9: Create a residuals plot for a regression model trained on this data:

X = [10, 20, 30, 40, 50]
Y = [15, 35, 40, 50, 65]

Python Code:

```
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt

X = np.array([10,20,30,40,50]).reshape(-1,1)
Y = np.array([15,35,40,50,65])

model = LinearRegression()
model.fit(X,Y)

pred = model.predict(X)
residuals = Y - pred

plt.scatter(pred,residuals)
plt.axhline(y=0, linestyle='--')
plt.show()
```



Assessment: The residuals appear moderately spread around zero. If the spread increases or decreases systematically, it may indicate heteroscedasticity.

Question 10: Imagine you are a data scientist working for a real estate company.

To address heteroscedasticity and multicollinearity in a house price prediction model, I would follow these steps:

1. Detect the Problems
• Use residual plots to detect heteroscedasticity.
• Use VIF (Variance Inflation Factor) to detect multicollinearity.

2. Handle Heteroscedasticity
• Apply logarithmic or square root transformations.
• Use weighted least squares regression.
• Remove influential outliers if necessary.

3. Handle Multicollinearity
• Remove highly correlated features.
• Combine correlated variables.
• Use dimensionality reduction techniques such as PCA.
• Apply Ridge or Lasso Regression.

4. Improve Model Performance
• Perform feature engineering.
• Split data into training and testing datasets.

• Evaluate using metrics such as R^2, MAE, RMSE, and MSE.

5. Validate the Model
• Use cross-validation to ensure the model generalizes well on unseen data.

These steps help create a stable, accurate, and robust regression model.